

■ IntelliAssist AI

Complete Project Documentation

AI-POWERED PDF ASSISTANT

A Streamlit application for uploading PDF documents and asking natural-language questions with responses generated through a local Ollama language model.

Prepared from the complete uploaded project ZIP

1. Project Overview

IntelliAssist AI is a Python and Streamlit-based document assistant. The project provides a web interface where users can upload a PDF, ask questions, and receive AI-generated answers. The main application combines Streamlit UI components, session-state conversation management, PDF text extraction, and a local ChatOllama model.

The complete project contains the main application, AI response module, PDF processing module, professional stylesheet, logo asset, backup application, data/upload directories, and dependency configuration.

2. Project Objectives

- Create an easy-to-use AI assistant for PDF documents.
- Allow users to upload PDF files through the Streamlit sidebar.
- Extract text from PDF pages.
- Allow natural-language questions about the uploaded document.
- Generate concise and professional AI responses.
- Maintain the current conversation in session state.
- Display recent questions in the sidebar.
- Provide a polished and responsive user interface.

3. Application Architecture

The project is divided into modules. `app.py` controls the interface and workflow. `pdf_chat.py` extracts text from PDFs. `chatbot.py` provides the AI response function. `assets/style.css` contains visual styling and `assets/logo.png` provides the logo.

User	→	Streamlit UI	→	PDF / Question
PDF	→	pdf_chat.py	→	Extracted Text
Question + Context	→	chatbot.py / ChatOllama	→	AI Answer
Answer	→	Session State	→	Conversation UI

4. Project File Structure

File / Folder	Purpose
app.py	Main Streamlit application, UI, session state, PDF upload, chat history and AI workflow.
chatbot.py	ChatOllama integration using model gemma:2b and response-generation rules.
pdf_chat.py	PDF text extraction using pypdf PdfReader.
utils.py	Present in the project; currently empty.
README.md	Present in the project; currently empty.
requirements.txt	Python dependency list.
assets/logo.png	Project logo asset.
assets/style.css	Professional UI stylesheet.
data/	Project data directory.

File / Folder	Purpose
uploads/	Project upload directory.
app_backup.py	Backup application source.

5. Main Application: app.py

Page configuration

The application configures the page title as IntelliAssist AI, uses a robot icon, a wide layout, and an expanded sidebar.

Session state

The application initializes message and chat-history collections in Streamlit session state. Messages represent the active conversation while chat history stores recent user questions.

PDF upload and processing

The sidebar provides a PDF uploader. When a file is present, app.py calls `read_pdf()` and displays the active document with a ready status.

Conversation interface

The application uses Streamlit chat components to display user and assistant messages. The professional interface includes user and AI avatars and a copyable response presentation.

Question processing

For PDF questions, extracted PDF content and the user's question are combined into a prompt instructing the model to answer only from the supplied PDF context.

Loading and Clear Chat

The AI call is wrapped in a loading spinner. Clear Chat resets both message and chat-history state and reruns the application.

6. AI Module: chatbot.py

The chatbot module initializes ChatOllama with the local gemma:2b model and temperature 0.3. The response function includes fixed responses for common greetings and identity questions, followed by a general prompt for other questions.

The prompt instructs IntelliAssist AI to answer professionally, stay concise, not identify itself as ChatGPT or Google AI, and use supplied PDF context when available.

Error handling

The model invocation is wrapped in exception handling and returns an error message if model invocation fails.

7. PDF Processing: pdf_chat.py

The PDF module uses pypdf's PdfReader. It creates a reader from the uploaded file, loops through all pages, extracts text where available, and concatenates the results.

Limitation

No OCR implementation is present in the supplied project, so scanned/image-only PDFs may require additional OCR support.

8. Professional UI and Styling

The project includes `assets/style.css` for the application's visual presentation. The stylesheet covers the application background, sidebar, headings, inputs, buttons, chat messages, avatars, animations, alerts, and responsive behavior.

- Professional sidebar and upload area.
- Rounded chat bubbles.
- Separate user and assistant message presentation.
- User and assistant avatars.
- Button hover effects.
- Message appearance animation.
- Responsive mobile layout.

9. User Features

- PDF upload and active document display.
- PDF-based question answering.
- Conversation history.
- Sidebar recent question history.
- User and AI avatars.
- Loading feedback during AI response generation.
- Copyable AI responses.
- Clear Chat.
- Responsive professional UI.

10. End-to-End Workflow

- Launch the Streamlit application.
- Upload a PDF.
- Extract PDF text page by page.
- Enter a natural-language question.
- Store the question in session state and chat history.
- Build a document-grounded prompt when a PDF is available.
- Send the prompt to ChatOllama.
- Generate the AI answer.
- Add the answer to the conversation.
- Display the updated conversation.

11. Dependencies

- `streamlit`
- `langchain`
- `langchain-community`

- langchain-core
- langchain-ollama
- python-dotenv
- pypdf
- faiss-cpu
- sentence-transformers

FAISS and sentence-transformers are declared in requirements.txt. The supplied current application files do not visibly implement an active FAISS/vector retrieval pipeline, so those packages are documented as declared dependencies rather than confirmed active features.

12. Current Project Status

The uploaded project contains the main Streamlit application, AI response module, PDF extraction module, styling asset, logo, backup source, and dependency file. README.md and utils.py are present but currently empty.

13. Testing Checklist

- ■ Start the Streamlit application.
- ■ Upload a valid PDF.
- ■ Confirm the PDF status card appears.
- ■ Ask a question answered by the PDF.
- ■ Confirm the answer appears in the conversation.
- ■ Confirm the question appears in Chat History.
- ■ Verify avatars and professional chat styling.
- ■ Verify the loading spinner.
- ■ Verify the copy control for an AI response.
- ■ Click Clear Chat and verify the state is reset.
- ■ Test scanned PDFs separately because OCR is not included.

14. Advantages

- Easy PDF question-answering workflow.
- Local AI model integration.
- Document-grounded prompt design.
- Professional UI foundation.
- Clear separation of PDF, AI and UI responsibilities.
- Session-based conversation handling.

15. Limitations

- No OCR support in the supplied implementation.
- Session history is not persistent database storage.
- Large PDFs may create very large prompts because full extracted text is inserted into the prompt.

- No authentication or multi-user system is shown.
- No source-page citation system is shown.
- FAISS and sentence-transformers are declared but not visibly used in the current workflow.
- README.md and utils.py are empty.

16. Future Enhancements

- Implement chunking, embeddings and retrieval for large PDFs.
- Use FAISS for semantic document retrieval.
- Use sentence-transformer embeddings.
- Add OCR for scanned documents.
- Add page/source citations to answers.
- Support multiple PDFs.
- Persist chat history in a database.
- Add authentication and user profiles.
- Add downloadable chat transcripts.
- Complete README.md with installation and usage instructions.

17. Conclusion

IntelliAssist AI is a practical AI-powered PDF assistant built with Python, Streamlit, pypdf and a local Ollama language model. The project combines PDF upload and extraction with document-grounded question answering and a professional conversational interface.

A future RAG-based retrieval layer would be the most valuable architectural improvement for handling larger documents efficiently.

18. Source Package Reviewed

- Gen AI chatbot/app.py
- Gen AI chatbot/app_backup.py
- Gen AI chatbot/chatbot.py
- Gen AI chatbot/pdf_chat.py
- Gen AI chatbot/utils.py
- Gen AI chatbot/README.md
- Gen AI chatbot/requirements.txt
- Gen AI chatbot/assets/style.css
- Gen AI chatbot/assets/logo.png
- Gen AI chatbot/data/
- Gen AI chatbot/uploads/

Documentation basis: This document is based on the uploaded project ZIP. Features not demonstrated by the source are not presented as implemented.